Step 14 - Exception Levels and Timer Ownership 1
Reference Note 1
Learning Objectives 1
1. ARM Exception Levels Overview 2
2. Why Multiple Timers Exist 2
3. Timer Ownership Model 3
4. Physical Timer 3
Registers 3
Purpose 3
Typical Owner 3
Architecture View 4
5. Virtual Timer 4
Registers 4
Purpose 4
Typical Owner 4
Architecture View 4
6. Hypervisor Timer 5
Registers 5
Purpose 5
Architecture View 5
7. Secure Physical Timer 5
Registers 6
Purpose 6
Architecture View 6
8. Shared Resource vs Owned Resource 6
Shared 6
Owned 7
9. Complete Ownership Table 7
10. Example Virtualized System 7

Guest Operating System 7
Hypervisor 8
Secure World 8
Why ARM Designed It This Way 8
Common Misconceptions 9
Misconception 1 9
Misconception 2 9
Misconception 3 9
Misconception 4 9
Architecture Summary 10
Key Takeaways 10
Knowledge Check 10
Q1 11
Q2 11
Q3 11
Q4 11
Q5 11
Answer Key 11

Step 14 - Exception Levels and Timer Ownership

Reference Note

This chapter is based on concepts described in the ARM Generic Timer Architecture and ARMv8-A architecture documentation.

Previous chapters introduced:

- Physical Counter (`CNTPCT_EL0`)
- Virtual Counter (`CNTVCT_EL0`)
- Physical Timer
- Virtual Timer
- `CNTVOFF_EL2`
- Access Control and Traps

This chapter explains:

Which Exception Level owns which timer?
Who programs which timer?
Why ARM provides multiple timers?

Learning Objectives

After completing this chapter, you should understand:

- Exception-level responsibilities
- Timer ownership model
- Physical Timer ownership
- Virtual Timer ownership
- Hypervisor Timer ownership
- Secure Timer ownership
- Why multiple timers exist

1. ARM Exception Levels Overview

Exception Level	Typical Role
EL0	Application
EL1	Operating System
EL2	Hypervisor
EL3	Secure Monitor

Not every system uses all exception levels.

Example:

System Type	Levels Used
Bare Metal	EL1
OS Only	EL0 + EL1
Virtualized	EL0 + EL1 + EL2
Secure System	EL0 + EL1 + EL2 + EL3

2. Why Multiple Timers Exist

Consider a virtualized system:

Software	Needs Timer?
Guest OS	Yes
Hypervisor	Yes
Secure Monitor	Yes

If only one timer existed:

Everyone would compete for the same timer.

ARM avoids this problem by providing separate timer resources.

3. Timer Ownership Model

Timer	Typical Owner
Physical Timer	EL1 OS
Virtual Timer	Guest OS
Hypervisor Timer	EL2 Hypervisor
Secure Physical Timer	EL3 Secure World

Think of these as:

Independent timer views
sharing the same System Counter.

4. Physical Timer

Registers

Register
CNTP_CVAL_EL0
CNTP_TVAL_EL0
CNTP_CTL_EL0

Purpose

Generate timer events using:

Physical Counter
(CNTPCT_EL0)

Typical Owner

Level	Usage
EL1	Primary User
EL2	May control/trap
EL3	Full access

Architecture View

Component	Used By
Physical Counter	EL1
Physical Timer	EL1

5. Virtual Timer

Registers

Register
CNTV_CVAL_EL0
CNTV_TVAL_EL0
CNTV_CTL_EL0

Purpose

Generate timer events using:

Virtual Counter
(CNTVCT_EL0)

Typical Owner

Level	Usage
Guest OS (EL1)	Primary User
EL2	Configures Virtual Time

Architecture View

Component	Used By
Virtual Counter	Guest
Virtual Timer	Guest

6. Hypervisor Timer

ARM provides a dedicated timer for EL2.

Registers

Register
CNTHP_CVAL_EL2
CNTHP_TVAL_EL2
CNTHP_CTL_EL2

Purpose

Generate timer events for:

Hypervisor Activities

Examples:

- vCPU scheduling
- VM management
- Hypervisor services

Architecture View

Component	Used By
Hypervisor Timer	EL2

7. Secure Physical Timer

Used by the Secure World.

Registers

Register
CNTPS_CVAL_EL1
CNTPS_TVAL_EL1
CNTPS_CTL_EL1

Purpose

Generate timer events for:

Secure Software
Trusted Firmware
Secure Monitor

Architecture View

Component	Used By
Secure Physical Timer	EL3 / Secure World

8. Shared Resource vs Owned Resource

Shared

These are shared by the entire system:

Component
System Counter
Counter Frequency

Owned

These are programmed by specific software layers:

Component	Owner
Physical Timer	EL1
Virtual Timer	Guest
Hypervisor Timer	EL2
Secure Timer	EL3

9. Complete Ownership Table

Resource	Typical Owner	Time Source
CNTPCT_EL0	EL1	Physical Time
CNTVCT_EL0	Guest EL1	Virtual Time
Physical Timer	EL1	Physical Counter
Virtual Timer	Guest EL1	Virtual Counter
Hypervisor Timer	EL2	Physical Counter
Secure Timer	EL3	Physical Counter

10. Example Virtualized System

Guest Operating System

Uses:

Resource
CNTVCT_EL0
CNTV_CVAL_EL0
CNTV_TVAL_EL0

Guest sees:

Virtual Time

Hypervisor

Uses:

Resource
CNTHP_*
CNTVOFF_EL2
CNTHCTL_EL2

Hypervisor manages:

Virtual Time
Guest Scheduling

Secure World

Uses:

Resource
CNTPS_*

Secure software receives:

Secure Timer Events

Why ARM Designed It This Way

Benefits:

Benefit	Description
Isolation	Each layer has its own timer
Efficiency	No software emulation needed
Virtualization	Guests receive independent timers
Security	Secure world has dedicated timer
Scalability	Multiple software layers coexist

Common Misconceptions

Misconception 1

Physical Timer belongs to EL0

False.

It is typically owned by the OS at EL1.

Misconception 2

Virtual Timer uses another clock source

False.

It ultimately derives time from the System Counter.

Misconception 3

Hypervisor uses Virtual Timer

Generally false.

The Hypervisor has its own:

Hypervisor Timer

Misconception 4

Each timer has its own counter

False.

All timers ultimately derive time from the same System Counter.

Architecture Summary

Shared Resources	Owner
System Counter	Entire System
Counter Frequency	Entire System

Timer Resource	Typical Owner
Physical Timer	EL1
Virtual Timer	Guest EL1
Hypervisor Timer	EL2
Secure Timer	EL3

Key Takeaways

Concept	Key Point
System Counter	Shared by all software layers
Physical Timer	EL1 timer
Virtual Timer	Guest timer
Hypervisor Timer	EL2 timer
Secure Timer	EL3 timer
Ownership	Separate timers, shared time source

Knowledge Check

Q1

Which timer typically belongs to EL1?

Q2

Which timer is used by guest operating systems?

Q3

Which timer is dedicated to EL2?

Q4

Which timer is used by the Secure World?

Q5

Do these timers use separate hardware counters?

Answer Key

Question	Answer
Q1	Physical Timer
Q2	Virtual Timer
Q3	Hypervisor Timer
Q4	Secure Physical Timer
Q5	No, all derive time from the System Counter